

Session 9 : Baking de Textures avec Blender

💡 Objectifs de la session

Partir d'un modèle généré par IA — diffuse acceptable, mais UVs non exploitables — et le rendre **propre** pour un moteur temps réel. On va voir comment **baker** (précalculer et transférer) une texture d'un UV vers un autre, refaire un dépliage soigné (marquage de seams, optimisation de la taille des îlots), puis ajouter du détail géométrique haute résolution et en **baker une normal map**. La session se termine par une **démonstration en direct** dans Blender, suivie d'une pratique d'éclairage statique dans Unity ou Godot (lightmaps + light probes pour un objet dynamique).

💡 Rappel : UVs, PBR et lightmaps

- **Session 7** : les UVs relient chaque sommet à un texel ; le dépliage produit des îlots séparés par des seams ; le set PBR (albedo, normal, roughness, metallic, AO) définit un matériau moderne.
- **Session 8** : les render targets permettent de rendre la scène dans une texture — c'est exactement le mécanisme qu'utilise le baking.
- **Session 6** : les lightmaps sont des textures d'éclairage précalculé, échantillonnées via un canal UV2 dédié ; les light probes fournissent l'éclairage baked aux objets dynamiques.

Partie 1 : Le point de départ — un modèle issu de l'IA

Le contexte

Les générateurs de modèles 3D par IA (Meshy, Tripo, Rodin, Luma, etc.) produisent désormais un mesh + un set de textures PBR en quelques secondes. Le résultat est souvent **visuellement correct** en diffusion (albedo), mais souffre de deux défauts majeurs pour un usage en moteur :

- **UVs non optimisées** : dépliage automatique sans logique d'îlots, étirement, chevauchements, résolution gaspillée.
- **Détail géométrique absent** : le mesh est low-poly, les détails visuels sont "peints" dans la diffuse — pas de vrai relief, pas de normal map cohérente.

💡 Pourquoi ce n'est pas utilisable tel quel

- **En moteur**, des UVs étirées donnent une texture déformée et des mipmaps qui clignotent.
- **Sans normal map**, l'objet paraît plat sous un éclairage dynamique — la diffuse "fait semblant" d'avoir du relief, mais la lumière ne réagit pas.
- **Le baking** est le pont entre ces deux états : on reconstruit un matériau propre à partir du matériau IA.

Partie 2 : Le Baking — Principe Général

Qu'est-ce que baker ?

Baker une texture, c'est **précalculer** le résultat d'un rendu et l'écrire dans un fichier image, au lieu de le calculer en temps réel. Concrètement, Blender rend la scène (ou une partie) dans un **render target** (Session 8), puis sauvegarde ce render target comme texture.

Deux grandes familles de baking :

- **Baking de transfert** : recalculer une texture existante pour qu'elle corresponde à un **nouvel ensemble d'UVs**. Le rendu est le même, seul le dépliage change.
- **Baking de détail** : capturer le détail d'un mesh **haute résolution** vers la texture d'un mesh **basse résolution** (normal map, déplacement, AO).

🗨 Le mécanisme commun — Cycles + Render Target

Dans Blender, le baking utilise le moteur **Cycles** (ray tracing, CPU/GPU). Pour chaque texel de la texture cible :

1. Le renderer trouve la face du mesh correspondante via les UVs cibles.
2. Il calcule la valeur demandée (couleur diffuse, normale, AO, etc.) à cet endroit.
3. Il écrit le résultat dans le pixel de la texture.

C'est exactement un **render target** (Session 8) — sauf qu'au lieu de rendre la vue d'une caméra, on rend **depuis les UVs** du mesh.

Bake Type	Sortie	Usage	Quand l'utiliser
Diffuse	Couleur RGB	Albedo propre	Transférer une diffuse vers de nouveaux UVs
Normal	Normales RGB	Relief apparent	Détail high-poly → low-poly
AO	Grayscale	Occlusion ambiante	Coin sombres, contact
Roughness / Metallic	Grayscale	PBR	Transférer le set PBR
Combined	RGB + éclairage	Lightmap	Précalculer l'éclairage d'une scène

Table 1: Les principaux **Bake Type** de Blender. On en utilise trois aujourd'hui : **Diffuse** (transfert), **Normal** (détail), et indirectement **Combined** pour les lightmaps de la pratique (Session 6).

Partie 3 : Baking de Diffuse — Transférer vers de nouveaux UVs

Le problème

Le modèle IA a une diffuse **correcte**, mais ses UVs sont mauvaises. Si on refait le dépliage, la diffuse ne correspond plus — chaque texel de l'ancienne texture pointait vers une face avec les anciennes UVs. Il faut donc **recopier** la diffuse dans une nouvelle texture organisée selon les **nouveaux UVs**.

1. Le workflow en deux meshes

🗨 L'astuce du double mesh

Le baking de transfert repose sur **deux meshes** :

- **Source** : le mesh d'origine avec ses UVs IA et sa diffuse. On le cache ou le désactive pour le rendu final.

- **Cible** : le même mesh (dupliqué), avec les **nouveaux** UVs et un matériau **vide** (Image Texture node branché sur le **Color** du Principled BSDF, avec une image vide créée).

Les deux meshes sont **superposés** (même position, même forme). Blender, pour chaque texel de la cible, va chercher la couleur correspondante sur la source par **ray casting** ou par correspondance d'UV.

2. Configuration dans Blender

Étapes du bake de diffuse.

1. **Sélectionner la source** (mesh IA + diffuse), puis **Shift-clic** la cible (le dernier sélectionné est l'objet actif = cible).
2. Ouvrir le **Shader Editor** sur la cible : ajouter un node **Image Texture**, créer une nouvelle image (ex: 2048×2048, 32-bit float si on veut garder de la plage), la brancher sur le **Base Color** du Principled BSDF. Garder ce node **sélectionné** — c'est lui qui reçoit le bake.
3. Dans **Render Properties** → **Bake** :
 - **Bake Type** : Diffuse.
 - **Influence** : décocher **Direct** et **Indirect** pour ne garder que **Color** (on veut l'albedo, pas l'éclairage).
 - **Margin** : 4-8 px (dilate la couleur dans les espaces entre îlots pour éviter des seams noirs au mipmapping).
4. Cliquer **Bake**. La nouvelle image se remplit, on l'enregistre (Image → Save As).

⚠ Pièges fréquents

- **Selected to Active** : c'est l'option qui dit "bake de la source sélectionnée vers la cible active". Si elle est cochée par erreur (ou inversement), le bake échoue ou produit du noir. Vérifier le sens : **source sélectionnée en premier, cible active en dernier**.
- **Cage / Extrusion** : si les deux meshes ne sont pas parfaitement superposés, des rayons peuvent rater la source. L'option **Cage** ou un **Extrusion** positif (ex: 0.05) donne une marge de tolérance.
- **Margin** : sans margin, les mipmaps affichent des bordures noires entre îlots en moteur — toujours mettre au moins 4 px.

Partie 4 : Refaire un UV Propre

L'objectif d'un bon dépliage

Un dépliage propre maximise la **qualité visuelle en moteur** pour un coût de texture donné :

- **Pas d'étirement** : une face 3D carrée doit occuper une zone 2D carrée.
- **Îlots logiques** : regrouper les faces par région du modèle (une île par bras, par jambe, par face de boîte) plutôt qu'en mille morceaux.
- **Packing compact** : minimiser l'espace vide — la texture est un bien précieux (VRAM).
- **Taille des îlots proportionnelle à leur visibilité** : une face qu'on voit souvent doit être plus grande dans la texture qu'une face cachée dessous.

1. Marquer les seams

Le marquage manuel — la base du bon dépliage

Le **Smart UV Project** de Blender (Session 7) fait un dépliage automatique, mais il produit beaucoup de petites îlots et des seams visibles. Pour un modèle destiné à un jeu, on **marque les seams à la main** en suivant des règles :

- **Coudre sur les arêtes cachées** : placer les seams là où l'œil ne les verra pas (dessous, dos, intérieur des bras, dessous des pieds). Un seam sur le devant du visage est catastrophique.
- **Suivre les lignes naturelles** : aisselles, ceinture, lignes de vêtements, bordures de pièces mécaniques. Le seam suit une couture réelle.
- **Minimiser le nombre d'îlots** sans créer d'étirement : une île par région logique, pas une île par face.

)

Workflow de marquage.

1. Mode Édition, sélection d'arêtes (2).
 2. Sélectionner une arête, Ctrl-E → Mark Seam (ou Right-click → Mark Seam selon la keymap).
- **Astuce** : sélectionner une **boucle d'arêtes** avec Alt-clic puis Mark Seam coupe tout un tour en une action.
1. Une fois les seams posées, **tout sélectionner** (A), U → Unwrap. Les îlots apparaissent dans l'UV Editor.
 2. Vérifier l'étirement avec l'overlay **Stretch** (UV Editor → View → Display Stretch) : les zones rouges sont étirées, il faut replacer un seam.

2. Optimiser la taille des îlots

Pourquoi la taille des îlots compte

La **densité de texels** d'une île = (aire de l'île dans la texture) / (aire 3D de la surface). Plus elle est élevée, plus la texture est nette sur cette région. Si toutes les îlots ont la même densité, la texture est **uniformément nette** — c'est l'idéal. Si une îlot occupe 50% de la texture pour une face cachée dessous, on gaspille de la VRAM.

Règles pratiques

- **Régions visibles en premier plan** (visage, poignée, plaque avant) : îlots **grandes**.
- **Régions rarement vues** (dessous, dos, intérieur) : îlots **petites**.
- **Uniformiser la densité** : dans l'UV Editor, **Select** → **All**, puis UV → **Average Island Scale** (ou Ctrl-A en mode UV) égalise la densité entre îlots.
- **Packing** : UV → **Pack Islands** (Ctrl-P) réarrange les îlots sans chevauchement. L'option **Margin** (ex: 0.01) laisse un espace pour le bake margin.
- **Aligner** : pour les îlots rectangulaires (faces de boîte, panneaux), les aligner sur les axes pour un packing dense.

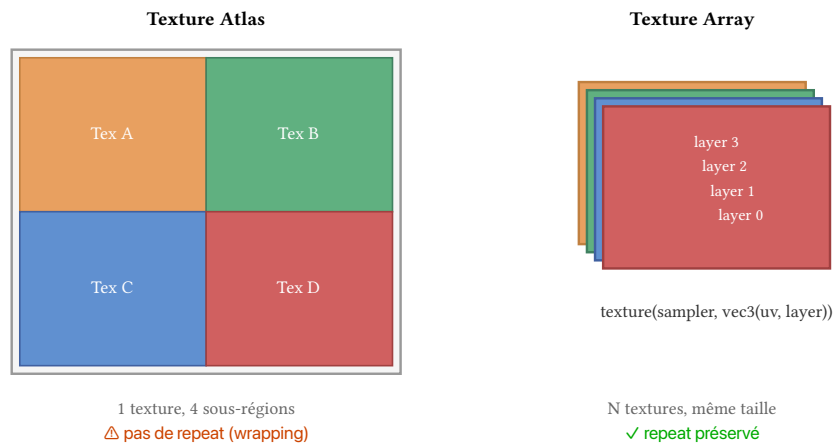


Figure 1: Bon vs mauvais packing. À gauche : îlots mal réparties, étirement, espace gaspillé. À droite : îlots logiques, densité uniforme, packing compact. La même texture 2048×2048 donne une qualité visuelle très différente selon le dépliage.

3. Itérer bake → dépliage → bake

⚠ Le baking n'est pas une étape finale

Une fois la diffuse bakée vers les nouveaux UVs, on peut découvrir que certains îlots sont trop petits (détail flou) ou mal orientés (motif qui “tourne” bizarrement). Le workflow est **itératif** :

1. Marquer les seams, déplier, packer.
2. Baker la diffuse.
3. Inspecter le résultat en moteur (ou en viewport Blender avec la nouvelle texture).
4. Ajuster les seams / la taille des îlots, rebaker.

Ne pas hésiter à **rebaker** — c’est une opération rapide (quelques secondes pour une diffuse 2K), et le gain de qualité est énorme.

Partie 5 : Baking de Normal Map — Détail High-Poly → Low-Poly

Le principe

Une **normal map** (Session 7) stocke des normales perturbées dans un RGB — elle **truque l’éclairage** pour faire croire qu’il y a du relief. Pour obtenir une normal map **cohérente avec la géométrie réelle**, on sculpte un mesh **haute résolution** (millions de polygones) et on bake ses normales vers la texture d’un mesh **basse résolution** (quelques milliers de polygones).

Le low-poly est utilisé en moteur (rapide à rendre), mais il **affiche** le détail du high-poly grâce à la normal map.

1. Sculpter le détail

💡 Workflow de sculpt

1. **Dupliquer** le mesh cible (low-poly) et le renommer high.
2. Passer en **Sculpt Mode**, ajouter des **Multiresolution Modifier** ou utiliser le **Dyntopo** pour sculpter des détails : rayures, boulons, écailles, stries, dommages.
3. On n’a pas besoin d’être photoréaliste — le détail sera **baked**, donc visible uniquement via la normal map. L’important est que les formes soient **lisibles** à l’échelle du modèle.

4. Conserver le low-poly **inchangé** : c'est lui qui sera exporté en moteur.

)

2. Baker la normal map

Étapes du bake de normal.

1. **Sélectionner le high-poly**, puis **Shift-clique** le low-poly (actif = cible).
2. Dans le Shader Editor du low-poly : ajouter un **Image Texture** node, créer une nouvelle image (2048×2048, 16-bit, **Non-Color** color space — important, pas de sRGB), le brancher sur le **Normal Map** node lui-même branché sur le **Normal** du Principled BSDF. Garder le node sélectionné.
3. Dans **Render Properties** → **Bake** :
 - **Bake Type** : Normal.
 - **Space** : Tangent (le standard pour les moteurs — Unity, Godot, Unreal l'attendent par défaut). Object est rare.
 - **Selected to Active** : **coché** (on bake du high vers le low).
 - **Extrusion** : 0.05 (marge pour le ray casting).
 - **Cage** : optionnel, utile si la forme est concave.
 - **Margin** : 4 px.
4. Cliquer **Bake**, sauvegarder l'image.

)

💡 Espaces tangents — cohérence moteur

Une normal map **tangent space** dépend de la **base tangente** du mesh (vecteurs tangent, bitangent, normale par sommet). Si le low-poly est modifié après le bake (ou exporté avec une autre option de normales), la normal map “clique” ou s'inverse. Règle d'or :

- **Baker après** avoir finalisé le low-poly (normales, smoothing, export).
- **Exporter** le low-poly avec les **normales calculées** (FBX/glTF avec +X forward, +Y up selon le moteur).
- En moteur, s'assurer que le matériau attend une normal map **tangent space** (défaut dans Unity/Godot).

⚠ Erreurs classiques de normal bake

- **Couleurs arc-en-ciel** : le ray casting a raté le high-poly par endroits → augmenter **Extrusion** ou utiliser un **Cage**.
- **Normales inversées** (bleu au lieu de rouge/vert) : mauvais **Space** ou low-poly dont les normales sont retournées (Ctrl-N → Recalculate Outside).
- **Seams visibles** : le low-poly a des **sharp edges** aux seams. Soit lisser les normales (Shade Smooth + **Auto Smooth**), soit baker avec un **Edge Split** désactivé.
- **sRGB au lieu de Non-Color** : la normal map est interprétée comme couleur → éclairage cassé. Vérifier le **Color Space** dans l'Image node et à l'import en moteur.

3. Compléter le set PBR

💡 Baker aussi l'AO et le roughness

Une fois la normal map bakée, on peut aussi baker :

- **AO** (Bake Type → Ambient Occlusion, Selected to Active) : capture l'occlusion du high-poly dans les creux — un AO baked est plus précis qu'un SSAO temps réel.
- **Roughness / Metallic** : si le high-poly a des matériaux assignés par face, on peut baker ces canaux aussi, ou les peindre manuellement dans l'Image Editor.

On se retrouve avec un **set PBR complet** (albedo + normal + AO + roughness + metallic) prêt pour un matériau Unity/Godot.

)

Partie 6 : Démonstration en Direct

💡 Déroulement de la démo

La démo se fait en **live** dans Blender, sur un modèle IA fourni (ex: un objet simple généré via Meshy ou Tripo — un bouclier, une caisse, un petit prop). Les étapes suivent exactement les Parties 3 à 5.

Plan de la démo (45-60 min)

Phase 1 — Diagnostic (5 min)

- Ouvrir le modèle IA, montrer la diffuse et les UVs dans l'UV Editor.
- Pointer les défauts : étirement, îlots en mille morceaux, chevauchements, seams au milieu du visage.

Phase 2 — Baking de transfert de diffuse (10 min)

- Dupliquer le mesh en source (UVs IA) et cible (UVs qu'on va refaire).
- Refaire le dépliage de la cible : marquer les seams à la main, Unwrap, Average Island Scale, Pack Islands.
- Configurer le bake (Diffuse, Color only, Selected to Active, Margin 6), baker, sauver la nouvelle diffuse.
- Comparer l'ancienne et la nouvelle texture dans l'UV Editor.

Phase 3 — Sculpt high-poly (15 min)

- Dupliquer la cible en high, passer en Sculpt Mode.
- Ajouter des détails : rayures, boulons, écailles, dommages — selon le prop.
- Insister sur le fait que le low-poly reste **inchangé**.

Phase 4 — Baking de normal map (10 min)

- Configurer le bake (Normal, Tangent, Selected to Active, Extrusion 0.05, Margin 4).
- Baker, sauver en Non-Color.
- Brancher la normal map dans le Principled BSDF via un **Normal Map** node, observer le relief en viewport (mode Material Preview ou Rendered).

Phase 5 — Bonus : AO baked (5 min)

- Baker l'Ambient Occlusion du high vers le low, la multiplier avec la diffuse dans le shader.

Phase 6 — Export et test en moteur (5-10 min)

- Exporter le low-poly en glTF/FBX + les textures (albedo, normal, AO).
- Ouvrir rapidement dans Godot ou Unity, assigner le matériau Standard, vérifier que la normal map réagit à la lumière.

⚠ Points à insister pendant la démo

- **L'ordre des sélections** source/cible — c'est l'erreur n°1 des étudiants.
- **Le Color Space** Non-Color de la normal map — sinon tout casse en moteur.
- **Le margin** — sans lui, des seams noirs apparaissent au mipmapping.
- **L'itération** — un premier bake est rarement parfait, on ajuste et on rebake.

Partie 7 : Pratique — Éclairage Statique en Moteur

💡 Objectif de la pratique

Mettre en application les concepts de la **Session 6** (lightmaps, light probes) : construire une petite scène statique, **baker son éclairage**, et éclairer un **objet dynamique** avec des **light probes**. Au choix : **Unity** ou **Godot**.

💬 Ce qu'on réutilise de la Session 6

- **Lightmap** : texture d'éclairage précalculée, échantillonnée via UV2 (Partie 2 de la Session 6).
- **Light probes** : échantillonnent l'éclairage baked à des positions, interpolées pour les objets dynamiques (Partie 4 de la Session 6).
- **Stratégie mixte** : statique en lightmap, dynamique en probes + lumières temps réel (Partie 5 de la Session 6).

Voir les figures et de la Session 6 pour rappel visuel.

)

Cahier des charges

La scène à construire

Une **pièce intérieure simple** (un sol, 4 murs, un plafond) avec :

- **Géométrie statique** : les murs, le sol, le plafond, et quelques props posés (caisses, chaise, table). Tout est marqué **static** / **bake mode : static**.
- **Une lumière directionnelle ou ponctuelle** par une fenêtre ou ouverture — source principale.
- **Une lumière secondaire** plus douce (lampe d'ambiance, ou sky light par la fenêtre).
- **Un objet dynamique** : un petit personnage (capsule, cube, ou le modèle IA de la démo) qui se **déplace** dans la pièce au runtime (script simple de patrol ou contrôle clavier).

)

Étapes — Godot

Workflow Godot (LightmapGI + LightmapProbe).

1. Créer la scène, placer les meshes statiques et la lumière.
2. Marquer les meshes statiques : **Geometry** → **GI Mode** → **Static**.
3. Ajouter un noeud **LightmapGI**, régler la **Bake Quality** sur **Medium** (pour la vitesse en classe).
4. Placer quelques **LightmapProbe** autour de la zone où l'objet dynamique se déplace.
5. Marquer l'objet dynamique : **GI Mode** → **Dynamic**.
6. **Bake** (bouton **Bake Lightmaps** du LightmapGI). Attendre la fin du calcul.
7. Lancer la scène, déplacer l'objet dynamique, observer :
 - Les murs et le sol sont éclairés par la lightmap (aucun calcul runtime).

- L'objet dynamique reçoit l'éclairage des probes — il s'assombrit dans les coins, s'éclaire près de la fenêtre, sans aucune lumière temps réel.

8. **Référence** : LightmapGI et LightmapProbe.

)

Étapes — Unity

Workflow Unity (Progressive Lightmapper + Light Probes).

1. Créer la scène, placer les meshes et la lumière.
2. Marquer les meshes statiques : cocher **Static** dans l'inspecteur (ou **Lightmap Static** dans le menu déroulant).
3. **Window** → **Rendering** → **Lighting**, onglet **Baked Lightmaps**.
4. **Lighting Mode** : **Baked Indirect** ou **Subtractive** (selon le résultat voulu). **Realtime Shadow Color** pour les ombres dynamiques en mode Subtractive.
5. Placer un **Light Probe Group** (GameObject → Light → Light Probe Group), étendre les probes autour de la zone de déplacement.
6. Cliquer **Generate Lighting** (en bas du panneau Lighting). Attendre la fin du bake.
7. L'objet dynamique reçoit automatiquement l'éclairage des probes (pas de réglage supplémentaire si le Renderer est standard).
8. Lancer, déplacer l'objet, observer la cohérence d'éclairage entre statique et dynamique.
9. **Référence** : Unity Lightmapping et Unity Light Probes.

)

Critères de réussite

Ce qui est évalué

- **Lightmap visible** : les murs et le sol montrent un éclairage baked (taches claires, ombres portées, AO dans les coins) — pas de calcul temps réel pour la géométrie statique.
- **Cohérence probes** : l'objet dynamique change d'éclairage selon sa position (assombrit dans les coins, éclairé près de la fenêtre) **sans lumière temps réel** — preuve que les probes fonctionnent.
- **Pas de fuite** : pas de "lumière qui traverse un mur" (signe d'une probe mal placée ou d'un mesh non statique).
- **Performance** : avec les lumières temps réel désactivées, la scène reste correctement éclairée — preuve que l'éclairage est bien baked.

)

⚠ Pièges fréquents en pratique

- **Mesh non marqué static** : la lightmap ne s'applique pas, l'objet reste noir ou éclairé en temps réel. Vérifier le **GI Mode** (Godot) ou **Static** (Unity) sur **tous** les meshes statiques.
- **Probes trop loin** : si l'objet dynamique sort de la zone couverte par les probes, l'éclairage devient plat ou erratique. Couvrir généreusement la zone de déplacement.
- **UV2 manquants** : Godot génère les UV2 automatiquement ; Unity aussi. Mais si un mesh importé a des UV2 explicites cassés, le bake échoue. Vérifier l'import.
- **Bake quality trop haute** : en classe, rester sur **Medium** — un bake **Ultra** sur une scène complexe peut prendre des dizaines de minutes.

)

💡 Bonus

- **Ajouter une reflection probe** (Session 6, Partie 6) près d'un objet métallique pour qu'il réfléchisse l'environnement baked.
- **Comparer** la même scène en **fully real-time** (SDFGI en Godot, Realtime GI en Unity) vs **baked** — mesurer la différence de FPS et de qualité.
- **Rebaker** après avoir déplacé une lumière : c'est l'inconvénient principal du baking, le vivre pour le retenir.

)